SIMATS ENGINEERING

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA12	**Course Title:**	Computer Architecture
CO Assessed:	CO5 – Precision (BL4)	**Assessment:**	Constraint Based Problem Solving
Weightage:	40%	**Total Marks:**	100
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	S Lakshmi priya	**Reg. No.:**	192524284
Section / Batch:	AI&DS	**Date:**	26/08/2026

InstructionstoStudents

- Identifytherelevantconceptsofl/Ointerfaces,interrupts,DMA,busarbitration,andhigh-speedcommunicationprotocols.
- AnalyzethelOrequirementsconsideringCPUutilization,latency,throughput,bandwidth,anddevicepriorities.
- ApplysuitablelOtechniquessuchasDMA,vectoredinterrupts,memory-mappedI/O,nestedinterrupts,andbusarbitration.
- Computeperformancemeasuressuchasdatatransferrate,interruptlatency,CPUutilization,throughput,andtransfertime.
- InterprettheresultsbyevaluatingCPUefficiency,communicationperformance,interruptresponse,andsuitabilityofthe proposed I/O architecture.

QUESTIONS

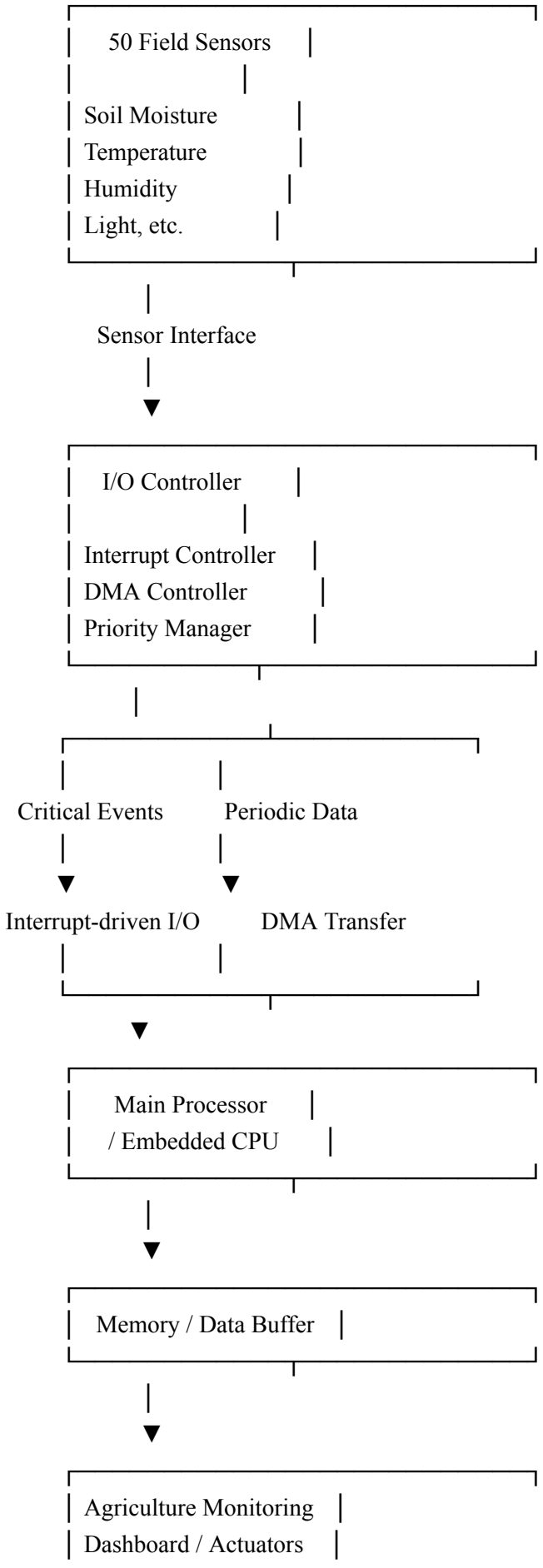
1. Design an I/O communication mechanism for a smart agriculture system with the following constraints:
   - 50 field sensors transmit readings at different intervals.
   - Soil-moisture alerts must be processed within 20 µs.
   - Use interrupt-driven I/O for critical events.
   - Use DMA for periodic bulk sensor-data transfer.
2. Construct an I/O subsystem for a video surveillance system with the following constraints:
   - 8 cameras continuously generate video streams.
   - The aggregate data rate is 1.5 GB/s.
   - Minimize CPU involvement during video transfer.
   - Use DMA with buffer management to prevent data loss.
3. Design a peripheral communication architecture for an industrial automation system with the following constraints:
   - 12 peripherals operate at different data rates.
   - High-priority control devices require deterministic response.
   - Low-priority devices must not monopolize the communication bus.
   - Implement priority-based bus arbitration.
4. Construct a data transfer mechanism for a real-time audio processing system with the following constraints:
   - Audio data must be transferred continuously at 200 MB/s.
   - Transfer interruptions must be minimized.
   - Use double buffering for continuous data processing.
   - Generate an interrupt only when a buffer transfer is completed.
5. Design an interrupt management system for a multicore embedded processor with the following constraints:
   - Support multiple interrupt sources simultaneously.
   - Distribute interrupts across processor cores to balance the workload.
   - Critical interrupts must have higher priority than routine interrupts.
   - Prevent excessive interrupt overhead during high system activity.
   - **Code of Conduct**

I S Lakshmi Priya(192524284)___(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: s priya_____

Design of I/O Communication Mechanism for Smart Agriculture System

1. System Architecture



2. Working Mechanism

- 50 sensors continuously collect soil moisture, temperature, humidity, and other field parameters.
- Sensors generate readings at different intervals, so a common input buffer is maintained.
- Soil-moisture alerts are treated as high-priority events.
- When soil moisture crosses a critical threshold, the sensor generates an interrupt.
- The interrupt controller immediately informs the CPU.
- The CPU saves its current state and executes the soil-moisture interrupt service routine.
- The alert must be processed within the required 20 μs.
- Normal periodic sensor readings are accumulated in a memory buffer.
- Instead of making the CPU handle every reading, the DMA controller transfers bulk sensor data directly between the I/O device and memory.
- After completing a DMA transfer, the DMA controller generates an interrupt to notify the CPU.
- The CPU processes the transferred data and updates the monitoring system.
- This combination gives fast response for critical events and efficient bulk data transfer for normal readings.

3. Priority Assignment

Event	I/O Method	Priority	Requirement
Soil-moisture alert	Interrupt	Very High	≤ 20 μs
Other critical sensor alert	Interrupt	High	Fast response
Periodic sensor readings	DMA	Medium	Bulk transfer
Dashboard update	Normal I/O	Low	Background

4. Pseudocode

```
START

Initialize 50 sensors
Initialize interrupt controller
Initialize DMA controller
Initialize sensor data buffer

Set soil-moisture alert threshold

Enable high-priority interrupts
Enable DMA

WHILE system is running

  IF soil-moisture interrupt occurs THEN
    Save CPU context

    Read soil-moisture value
    Check alert threshold

    IF moisture is below threshold THEN
      Generate irrigation alert
      Activate irrigation system if required
```

```
        END IF

        Clear interrupt flag
        Restore CPU context
    END IF


    IF DMA transfer is required THEN
        Configure DMA source
        Configure DMA destination
        Set transfer size

        Start DMA transfer

        DMA transfers sensor data
        directly to memory buffer

        IF DMA transfer completes THEN
            Generate DMA interrupt
            Process received sensor data
            Clear DMA interrupt
        END IF
    END IF

END WHILE

STOP
```

5. Meeting the 20 μs Requirement

To ensure the soil-moisture alert is processed within 20 μs:

1. Assign soil-moisture interrupts the highest priority.
2. Use interrupt-driven I/O instead of polling.
3. Keep the interrupt service routine short.
4. Store the sensor reading immediately.
5. Avoid lengthy processing inside the ISR.
6. Use DMA for normal sensor transfers so the CPU remains available.
7. Allow the high-priority interrupt to pre-empt lower-priority tasks.
8. Measure interrupt latency and ensure:

Interrupt detection + ISR execution ≤ 20 μs

6. Advantages

- Fast response: Critical soil-moisture events are handled immediately.
- Reduced CPU load: DMA handles periodic bulk transfers.
- Efficient communication: 50 sensors can operate at different sampling intervals.
- Priority handling: Critical agricultural alerts receive immediate attention.
- Scalable: Additional sensors can be added without significantly increasing CPU workload.
- Reliable monitoring: Sensor data is continuously collected and stored in memory.

Conclusion

The proposed mechanism combines priority-based interrupt-driven I/O for critical soil-moisture alerts with DMA for periodic bulk sensor-data transfers. This ensures that urgent alerts are processed within the 20 μs deadline, while DMA efficiently handles data from all 50 field sensors without continuously occupying the CPU.

2 **I/O Subsystem for Video Surveillance**

Architecture

8 Cameras

↓

DMA Controller

↓

Buffer A ↔ Buffer B ↔ Buffer C

↓

Main Memory

↓

CPU / Video Processing

Working

- **8 cameras** continuously send video data at an aggregate rate of **1.5 GB/s**.
- **DMA** transfers video directly from cameras to memory, reducing CPU involvement.
- Multiple buffers are used in **circular buffer management**.
- While DMA fills one buffer, the CPU processes another buffer.
- When a buffer is full, DMA automatically switches to the next available buffer.
- CPU is interrupted only when a buffer requires processing.
- If a buffer is full, it is marked **READY**; after processing, it becomes **FREE**.
- This prevents buffer overflow and **data loss**.

Pseudocode

```
Initialize DMA
Initialize Buffers A, B, C

WHILE cameras are active
   DMA transfers video to FREE buffer

   IF buffer is FULL
      Mark buffer READY
      Switch DMA to next FREE buffer
      Process READY buffer
      Mark buffer FREE
   END IF
END WHILE
```

Result

DMA + circular buffer management provides continuous **1.5 GB/s** video transfer with minimal CPU involvement and prevents data loss.

3 **Peripheral Communication Architecture – Industrial Automation**

Architecture

12 Peripherals

↓

Priority-Based Bus Arbiter

↓

Shared Communication Bus

↓

CPU / Controller

Working

- 12 peripherals operate at different data rates.
- Each peripheral sends a **bus request** when it needs communication.
- The **bus arbiter** checks the priority of all requests.
- High-priority control devices get **immediate and deterministic access**.
- Low-priority devices use **priority aging** to prevent starvation.
- After transfer, the peripheral releases the bus.

Pseudocode

```
Initialize 12 peripherals

WHILE system is running
  Check all bus requests

  Increase priority of waiting devices

  Select device with highest priority

  Grant bus access

  Perform data transfer

  Release bus
END WHILE
```

Result

Priority-based bus arbitration provides fast response to critical devices while ensuring that low-priority peripherals also get fair access to the communication bus.

4 **Data Transfer Mechanism – Real-Time Audio Processing**

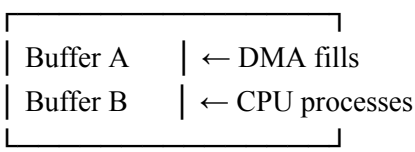
Architecture

Audio Input (200 MB/s)

↓

DMA Controller

↓



↓

Audio Processor

Working

- Audio data is transferred continuously at **200 MB/s** using DMA.
- **Double buffering** is used: while DMA fills Buffer A, CPU processes Buffer B.

- When Buffer A is full, DMA switches to Buffer B.
- CPU then processes Buffer A.
- An **interrupt is generated** only after a complete buffer transfer.
- This minimizes CPU interruptions and prevents data loss.

Pseudocode

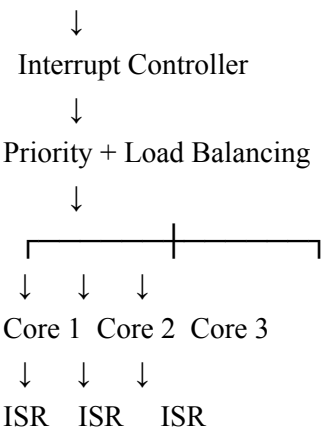
```
START
Initialize DMA
Initialize Buffer A and Buffer B

Start DMA → Buffer A

WHILE audio transfer is active

  IF Buffer A is full THEN
     Generate interrupt
     Start DMA → Buffer B
     Process Buffer A
  END IF

  IF Buffer B is full THEN
     Generate interrupt
     Start DMA → Buffer A
     Process Buffer B
  END IF

END WHILE
STOP
```

Result

DMA + double buffering provides continuous 200 MB/s audio transfer with minimal interruptions and efficient real-time processing.

5 Interrupt Management System – Multicore Processor

Architecture



Working

- Multiple interrupt sources send requests to the interrupt controller.

- Critical interrupts are assigned higher priority.
- The controller distributes interrupts among available CPU cores.
- Workload is balanced by assigning new interrupts to the least-loaded core.
- Routine interrupts are processed with lower priority.
- Interrupt coalescing can combine multiple routine interrupts to reduce overhead.
- Critical interrupts are handled immediately without waiting for routine interrupts.

Pseudocode

START

Initialize interrupt controller and CPU cores

WHILE system is running

  Receive interrupt requests

  Assign priority to each interrupt

  IF critical interrupt exists THEN

    Assign to highest-priority available core

    Service interrupt

  ELSE

    Select least-loaded core

    Assign routine interrupt

  END IF

  Combine routine interrupts when activity is high

  Clear completed interrupts

END WHILE

STOP

Result

The system provides priority-based interrupt handling, balances interrupts across multiple cores, and reduces interrupt overhead during high system activity.

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding ; misses key constraints	Fails to identify constraints correctly

Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Q. No	Problem Understanding (25)	Constraint Analysis (25)	Solution Development (25)	Application of Concepts (15)	Justification (10)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	**/25**	**/ 25**	**/ 15**	**/ 10**	**/ 100**

Course Coordinator **Faculty Incharge**